
Poisoning Attacks on LLM-Based Agents: A General Evaluation Framework

Zachary Bonagura
Rensselaer Polytechnic Institute
Troy, NY 12180
bonagz@rpi.edu

Abstract

Large language model (LLM) agents have emerged as powerful systems capable of reasoning, planning, and interacting with external tools and memory to perform complex tasks. However, their expanded capabilities also introduce new security vulnerabilities beyond traditional language models. In this work, we present an extensible framework for evaluating poisoning attacks on LLM-based agents, focusing on runtime manipulation rather than compromising at the model-level. We formalize a unified threat model in which an adversary operates in a black-box setting and injects malicious content through multiple agent interaction surfaces. Within this framework, we introduce a taxonomy of poisoning attacks targeting these components, including prompt injection, reasoning manipulation, tool interface poisoning, memory retrieval bias, and action-policy attacks, covering a wide range of agent capabilities. Each attack is implemented under a common interface, enabling systematic comparison across attack types and poisoning scopes. We further distinguish between single-instance and persistent poisoning, where persistent attacks store malicious content in long-term memory and influence future agent behavior through retrieval. To evaluate these attacks, we define metrics such as Attack Success Rate (ASR), Persistence Rate (PR), Task Accuracy (TA), and Refusal Rate (RR), and provide an experimental pipeline for benchmarking attacks both independently and in pairwise combinations across different agent configurations. Our results show that runtime poisoning can substantially compromise LLM-agent behavior even without access to model weights or training data. In single-surface experiments, prompt injection achieves the strongest immediate attack success, reaching 95.6% ASR, while memory retrieval and action-policy attacks show the strongest persistence, reaching 75.6% and 73.3% PR respectively. In combined experiments, several pairwise attacks compose successfully, with Prompt + Tool reaching 100.0% strict combined ASR and Reasoning + Action reaching 93.3% strict combined PR. Full-sweep attacks further reduce task accuracy to 0.0% across all evaluated scenario families, even when not every individual attack objective succeeds simultaneously. These findings show that LLM-agent vulnerabilities depend not only on isolated attack surfaces, but also on how poisoned prompts, reasoning directives, tool context, memory, and action policies interact within the broader agent pipeline.

1 Introduction

Large language models (LLMs) have rapidly evolved in recent years from standalone text generators into complex agentic systems capable of interacting with external tools, maintaining persistent memory, and performing multi-step reasoning. These LLM-based agents extend the capabilities of traditional models by incorporating additional components such as retrieval-augmented memory, tool

interfaces, and structured prompting pipelines. While these enhancements enable more powerful and flexible behavior, they also significantly expand the attack surface of such systems.

Recent work has demonstrated that LLMs are vulnerable to several forms of adversarial manipulation, including prompt injection, Trojan triggers, reasoning backdoors, retrieval poisoning, and agent-level backdoors [7, 8, 6, 1, 9]. However, existing studies often evaluate attacks against isolated prompts or model behaviors, but do not provide a unified framework for comparing several attacks across multiple interacting components of an LLM-based agent. In practice, agents operate across multiple interaction surfaces, including user inputs, system prompts, external tools, and long-term memory stores, which prior work has identified as important sources of agent-level security risk [9, 10, 1, 11]. This introduces new opportunities for adversaries to influence agent behavior at runtime without modifying model parameters or training data.

In this work, we study *runtime poisoning attacks* on LLM-based agents, where an adversary injects malicious content into the agent’s execution pipeline during inference. We consider a realistic black-box threat model in which the attacker cannot access model weights or internal states, but can manipulate inputs to various agent components. Unlike traditional data poisoning attacks, which occur during training, runtime poisoning targets the agent’s decision-making process as it executes tasks.

We propose a unified and extensible framework for evaluating poisoning attacks across different components of an LLM agent. Our framework introduces a taxonomy of attacks that target distinct interaction surfaces, including prompt injection, reasoning manipulation, tool interface poisoning, memory retrieval bias, and action-policy attacks. These attacks are implemented under a common interface, enabling systematic comparison across attack types and allowing new attack strategies to be easily incorporated.

A key distinction in our framework is between *single-instance* and *persistent* poisoning. Single-instance attacks affect only a single execution of the agent, while persistent attacks store malicious content in the agent’s long-term memory, enabling the attack to influence future interactions through retrieval. This distinction captures an important class of long-term vulnerabilities unique to agent systems.

To evaluate the effectiveness of these attacks, we define a set of evaluation metrics including Attack Success Rate (ASR), which measures the frequency of successful attack outcomes, and Persistence Rate (PR), which quantifies the ability of *persistent* attacks to influence agent behavior across fresh agent sessions through memory retrieval. In addition, we incorporate Task Accuracy (TA) to measure correctness of the underlying task and Refusal Rate (RR) to capture safety-related refusal behavior. We further provide a reproducible experimental pipeline that supports both single-surface evaluation and combined attack evaluation, allowing attacks to be studied independently or composed pairwise to test whether they amplify or interfere with one another.

Our contributions are as follows:

- We introduce a unified threat model for runtime poisoning attacks on LLM-based agents in a black-box setting.
- We propose a modular framework for evaluating attacks across multiple agent interaction surfaces, including prompt, reasoning, tool-interface, memory-retrieval, and action-policy components.
- We distinguish between single-instance and persistent poisoning, allowing immediate attack success and cross-session persistence to be evaluated separately.
- We formalize evaluation metrics that capture both adversarial effectiveness and task-level side effects, including ASR, PR, TA, and RR.
- We provide an extensible experimental pipeline for evaluating attacks independently and in combinations.

2 Background

2.1 LLM-Based Agents

Large language model (LLM) agents extend traditional language models by integrating additional components that enable interaction with external systems and multi-step reasoning. Unlike standalone LLMs, which generate responses based solely on a fixed prompt, agents operate within structured pipelines that may include system prompts, user input, intermediate reasoning steps, and external resources.

A typical LLM agent consists of several key components: (1) a prompt construction mechanism that combines system instructions and user inputs, (2) a reasoning process that may involve intermediate steps or an action selection policy, (3) access to external tools for performing actions such as retrieval or computation, and (4) a memory module that stores and retrieves information across interactions. These components allow agents to perform complex tasks but also introduce additional points of vulnerability.

2.2 Memory and Retrieval

Many LLM agents incorporate retrieval-augmented memory, where past interactions or external knowledge are stored and later retrieved to inform future responses. This memory is typically represented as vector embeddings or a collection of key-value pairs or text entries. Retrieval mechanisms may include full retrieval, top- k selection, key-based filtering, or random sampling.

The use of persistent memory enables agents to maintain context across sessions, improving long-term coherence and personalization. However, it also introduces a new attack surface, as adversaries may inject malicious content into the memory store that is later retrieved and influences future behavior [1, 2].

2.3 Tool Use in LLM Agents

Modern LLM agents frequently interact with external tools, such as APIs, databases, or computational modules. Tools are typically described to the model using natural language specifications, and the model decides when and how to invoke them. The outputs of these tools are then incorporated into the agent’s reasoning process to produce a final response.

While tool integration enhances agent capabilities, it also creates opportunities for manipulation, especially when adversarial context influences tool selection or interpretation of tool outputs [9, 10, 5]. Because tool descriptions and outputs are processed as text, adversaries can alter how tools are interpreted or used, potentially causing the agent to produce incorrect or malicious outputs.

2.4 Threat Model and Poisoning Attacks

Security research on language models typically considers adversaries that attempt to influence model behavior through adversarial inputs. In the context of LLM-based agents, poisoning attacks may occur either during training (data poisoning) or during inference (runtime manipulation).

Runtime poisoning attacks target the inputs and intermediate representations used by the agent during execution, such as prompts, retrieved memory, or tool interactions. These attacks can be transient, affecting only a single execution, or persistent, where malicious content is stored and later reused to influence future behavior. Many runtime poisoning attacks exhibit backdoor-like behavior in the sense that malicious behavior is activated conditionally by a trigger or contextual cue [8, 6, 9].

Understanding these threat models is essential for analyzing vulnerabilities in agent systems, as the combination of multiple components introduces new opportunities for adversarial manipulation

3 Related Work

Prompt, Reasoning, and Backdoor Attacks on LLMs

Recent work has shown that large language models are vulnerable to adversarial manipulation through prompts, triggers, and poisoned reasoning context. Early studies on prompt injection demonstrated that carefully crafted inputs can induce goal hijacking and prompt leakage in black-box language models, highlighting how fragile instruction-following systems are under adversarial prompt composition [7]. TrojLLM further showed that black-box Trojan prompt attacks can generate universal and stealthy triggers that reliably manipulate model outputs without requiring access to model parameters or training data [8].

A closely related area of work studies reasoning backdoors. BadChain shows that chain-of-thought prompting can itself become an attack surface by inserting malicious reasoning steps that alter model behavior under trigger conditions [6]. More recent work such as DarkMind and ShadowCoT explores latent or stealthy manipulation of reasoning processes, showing that intermediate reasoning itself can serve as a backdoor channel rather than merely the final text output [3, 12]. Together, these works motivate our inclusion of prompt-level and reasoning-level manipulations as representative runtime poisoning surfaces. However, our goal is not to introduce a new prompt or reasoning backdoor mechanism, but to incorporate these broad attack ideas into a common agent-centric evaluation framework.

Security Threats in LLM-Based Agents

As LLMs have evolved into tool-using and memory-enabled agents, recent research has extended these concerns from standalone language models to full agentic systems. *Watch Out for Your Agents!* explicitly investigates backdoor threats to LLM-based agents and frames agent backdoors as a distinct security problem with richer attack surfaces than traditional LLM attacks [9]. In particular, it highlights that malicious behavior in agents may be triggered not only through user inputs, but also through intermediate observations and reasoning steps during agent execution. Recent work has also showed that adversarial context can hijack the action plans of black-box LLM-based agents, highlighting that attacks may target not only final outputs but also the intermediate policy used to select executable actions [10].

Other research has examined poisoning attacks on retrieval and memory mechanisms. AgentPoison shows that poisoning long-term memory or retrieval sources can reliably induce malicious downstream behavior while preserving benign performance on clean inputs [1]. TrojanRAG similarly demonstrates that retrieval-augmented generation pipelines can themselves act as backdoor drivers, where poisoned retrieved content causes malicious behavior after retrieval [2]. At a broader system level, *The Dark Side of LLMs* studies compromise scenarios spanning direct prompt injection, RAG backdoors, and trust exploitation in multi-agent systems [5].

Together, these works show that adversarial influence in agents can arise through multiple components of the agent pipeline, including prompts, memory, reasoning, tools, and action selection. Our framework is motivated by this same observation, but differs in emphasis. Rather than proposing a new poisoning primitive or reproducing any one prior attack exactly, we unify representative runtime poisoning strategies under a common interface and support comparisons across poisoning scopes and attack combinations.

Comparison to Existing Benchmark Frameworks

Recent benchmark work has explored agent security at a broader system level. *AgentLAB* focuses on adaptive long-horizon attacks, introducing five attack families across 28 realistic agent environments and 644 security test cases to evaluate how vulnerabilities emerge over extended multi-turn interactions [4]. *Agent Security Bench (ASB)* provides a comprehensive framework for formalizing and benchmarking attacks and defenses in LLM-based agents, covering 10 scenarios, 10 agents, over 400 tools and tasks, 27 attack and defense methods, and 7 evaluation metrics [11].

Our framework shares the same overall motivation of systematic evaluation of vulnerabilities across multiple agent attack surfaces, but differs in design goal. Whereas AgentLAB and ASB emphasize large-scale, scenario-driven evaluation across diverse environments, agents, tools and attack settings,

our framework is designed to support controlled comparison of runtime poisoning mechanisms under a common abstraction. It centers on shared attacker assumptions, a unified attack interface, and explicit comparison across both attack surfaces and poisoning scopes, including single-instance and persistent settings. As a result, these benchmark frameworks provide broad system-level realism, while our framework contributes a lightweight and extensible pipeline for studying poisoning attacks under a shared attack interface and comparison across poisoning surfaces and scopes.

4 Framework

4.1 Overview

We present a general framework for evaluating runtime poisoning attacks on LLM-based agents. The primary goal of the framework is not to introduce entirely new poisoning primitives, but to provide a unified and extensible experimental setting in which representative attack classes can be implemented, combined, and evaluated under a shared interface. This differs from broader agent-security benchmarks that emphasize large-scale scenario coverage, many tools, and diverse attack environments [11, 4]. The framework is designed around the observation that modern LLM agents expose multiple interaction surfaces during execution, including user prompts, system instructions, retrieved memory, tool interfaces, intermediate reasoning, and action-selection guidance.

At a high level, the framework consists of four main components: (1) an execution context that represents all information available to the agent at runtime, (2) an abstract attack interface that defines how poisoning strategies interact with the agent, (3) an agent runner that executes the agent under clean or poisoned conditions, and (4) an experiment runner that performs evaluation across multiple contexts, attack settings, and trials. This design allows attacks to be studied both in isolation and in pairwise combinations, while keeping the evaluation pipeline consistent across different attack types. As a result, the framework can evaluate not only individual attack surfaces, but also interactions between attacks that may amplify or interfere with one another.

The primary goal of this framework is not to maximize attack success through highly optimized attack-generation procedures, but to enable controlled comparison of runtime poisoning mechanisms across different agent interaction surfaces. Prior work often incorporates attack-specific optimizations, such as trigger search, embedding-level manipulation, or demonstration selection, which can significantly increase attack success rates [8, 6, 1]. However, these techniques introduce additional variability that makes it difficult to isolate performance differences to the targeted component itself. In particular, if each attack is independently optimized, observed differences may reflect effectiveness of the optimization procedure rather than the vulnerability of the agent component being poisoned.

In contrast, our framework uses fixed, manually constructed attack instances to control for attack-generation variability and isolate the causal effect of poisoning at each runtime interface. Under this design, all attacks are instantiated under a shared structure and evaluated within a consistent execution pipeline, allowing direct comparison across surfaces and poisoning scopes. As a result, the framework should be interpreted as a mechanism-level evaluation of runtime poisoning behavior rather than a benchmark for maximum attack strength. This approach provides a reproducible baseline for understanding how different components of LLM-based agents respond to adversarial manipulation. Figure 1 illustrates the overall framework architecture, including the shared attack interface, runtime context, agent runner, tool loop, and persistent memory store.

4.2 Agent Model

We model an LLM-based agent as a structured execution pipeline that operates over a runtime context. At each execution step, the agent receives a system prompt, user input, retrieved memory, optional tool descriptions, and optional action-policy guidance. These components are assembled into a prompt template and passed to the underlying language model, which produces either a natural language response or a structured tool invocation.

The agent may access external tools through natural-language tool descriptions and JSON-style tool schemas. If the model emits a valid tool call, the corresponding tool is executed and its result is appended to the agent’s scratchpad before the model is queried again. In this way, the framework captures the common structure of tool-augmented agents in which reasoning, tool use, and memory retrieval occur within a single iterative execution loop. Persistent memory is modeled separately

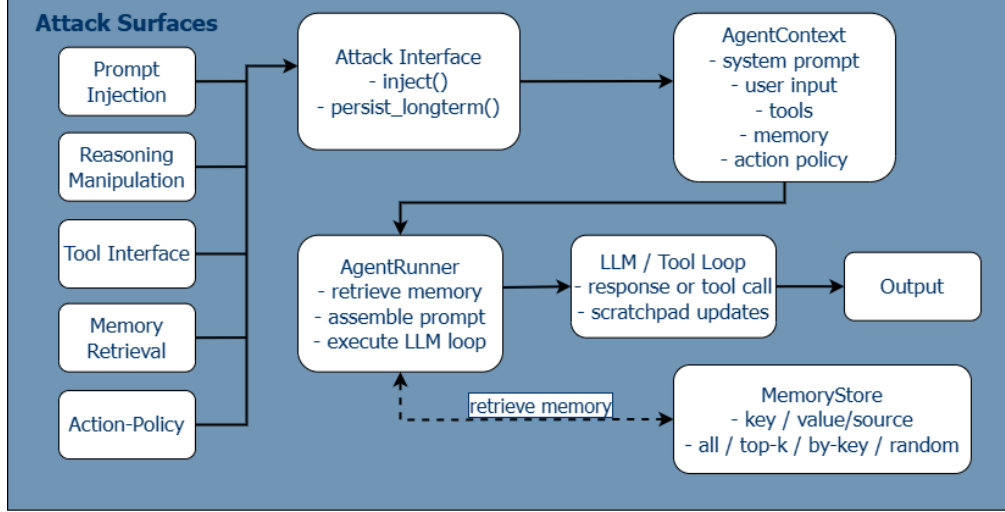


Figure 1: Framework architecture. Runtime poisoning attacks are implemented through a shared attack interface and modify the agent context before execution. The agent runner retrieves memory, assembles the prompt, executes the LLM/tool loop, and produces an output. Persistent memory is stored externally and retrieved according to the configured retrieval policy.

from the immediate execution context and is retrieved at runtime according to a configurable retrieval policy. The implementation follows a LangChain-style agent structure, in which prompts, tool descriptions, intermediate tool observations, and memory context are assembled into a structured execution loop before each model call.

The execution context is represented explicitly through an `AgentContext` object, which stores the label of the task instance together with the system prompt, user input, available tools, retrieved memory, optional action-policy directives, and metadata. This representation provides a unified interface over the information available to the agent at inference time and serves as the primary object manipulated by attacks.

4.3 Threat Model

We consider a black-box adversary who cannot modify model weights, internal hidden states, or the agent’s source code. Instead, the attacker influences the agent through runtime-accessible interaction surfaces. These surfaces include prompt inputs, intermediate reasoning guidance, tool descriptions and outputs, retrieved memory, and policy-like guidance that affects action selection.

The adversary may introduce malicious content either transiently within a single session or persistently through long-term memory. In the single-instance setting, the attack affects only the current execution. In the persistent setting, malicious content is written into the memory store and later retrieved in future sessions. The attacker may also exploit trigger conditions, where an attack activates only when a specific word or phrase is present.

This threat model is intentionally focused on runtime poisoning rather than compromising the training time of an agent. The goal is to study how poisoning behavior propagates through the agent pipeline under a shared and reproducible set of attacker assumptions.

4.4 Attack Abstraction

All attacks in the framework inherit from a common abstract `Attack` interface. Each attack is defined by a name, a target component, a success checker, a poisoning scope, and an optional trigger. The target component identifies the attack surface being manipulated, such as prompt construction, reasoning, tool interface, memory retrieval, or action-policy guidance.

The core method of the abstraction is `inject()`, which modifies the execution context before the agent runs. Persistent attacks may additionally implement `persist_longterm()`, which writes

malicious content into an external memory store. The abstraction also exposes optional hooks for modifying tool descriptions and judge metadata for LLM-based evaluation. This design allows attacks with different mechanisms to be expressed within a single interface while preserving enough flexibility to capture component-specific behavior.

Using this abstraction, the framework currently supports representative attack classes including prompt injection, chain-of-thought manipulation, tool interface poisoning, memory retrieval attacks, and action-policy attacks. Although these attacks differ in how they influence the agent, they are all evaluated through the same runtime pipeline and structure.

4.5 Poisoning Scopes

A central distinction in the framework is between *single-instance* and *persistent* poisoning. Single-instance attacks are active only during the current execution. They inject malicious content into the runtime context and are discarded once the session ends. This models temporary manipulation of different components of the agent.

Persistent attacks, by contrast, are designed to survive beyond the initial compromised session. In our implementation, a persistent attack first succeeds in the current session through active injection into the agent, and then stores malicious content into long-term memory. In a later fresh session, the attack is evaluated again without active injection, so that any malicious behavior must arise from retrieved poisoned memory alone. This separation allows the framework to distinguish immediate attack success from durability across sessions.

The memory store itself is represented as an external persistent collection of entries containing a key, value, and source label. Retrieval can be configured using multiple modes, including full retrieval, top- k selection, key-based filtering, and random sampling. This abstraction is intentionally lightweight and stored using JSON structure, allowing the framework to study persistent poisoning behavior without committing to a single memory architecture.

4.6 Experimental Pipeline

Experiments are managed by an `ExperimentRunner`, which manages memory reset, attack injection, evaluation, and result logging. For a clean baseline, the agent is evaluated on a set of contexts without any attack. For attack experiments, the framework rebuilds attacks for each trial, resets memory, and runs evaluation under the specified poisoning setting. Figure 2 shows how the experiment runner separates clean baseline evaluation, single-instance attack evaluation, and persistent attack evaluation.

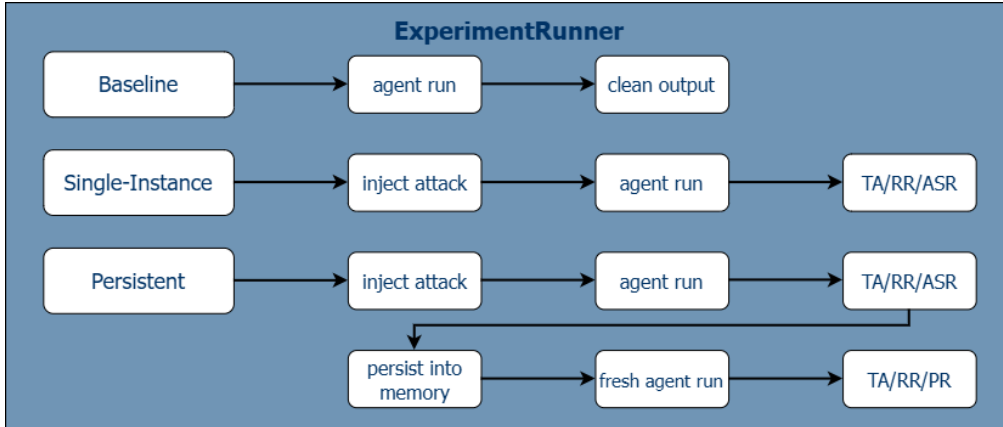


Figure 2: Evaluation pipeline. Baseline runs measure clean task performance. Single-instance attacks are evaluated through active injection followed by an attacked agent run. Persistent attacks are first evaluated under active injection for ASR, then written into memory and tested again in a fresh agent session for PR.

For single-instance attacks, the pipeline injects attacks directly into the current execution context and measures attack success on the resulting outputs. For persistent attacks, the pipeline first evaluates

attack success in the actively compromised session, then stores the attack into long-term memory, and finally launches a fresh agent session to measure whether the malicious behavior persists through memory retrieval alone. Results are written as structured JSONL records containing evaluation type, context label, attack metadata, success indicator, and agent output.

This design supports both isolated and combined attack studies. In the single-surface setting, one attack family is evaluated at a time to isolate the effect of a specific poisoning mechanism. In the combined setting, multiple attacks can be applied to the same execution context, enabling pairwise comparisons between attack A alone, attack B alone, and both attacks together. Because all attacks share the same experimental interface, multiple attacks can be composed in a single run while preserving a consistent execution and logging procedure.

4.7 Evaluation Metrics

We use a set of evaluation metrics: *Attack Success Rate (ASR)*, *Persistence Rate (PR)*, *Task Accuracy (TA)*, and *Refusal Rate (RR)*.

ASR measures the fraction of evaluation instances for which the attack succeeds during the actively attacked session. For trigger-conditioned attacks, we also track Trigger Rate (TR), the fraction of evaluation instances in which the attack condition is activated. ASR is computed over triggered attack instances rather than all evaluated contexts, preventing non-triggered examples from artificially lowering attack success.

PR is defined only for persistent attacks. It measures the fraction of fresh-session evaluations in which the persistent subset of attacks remains effective after malicious content has been written to long-term memory and active injection is removed. This way, ASR captures immediate attack effectiveness, while PR captures the durability of poisoned behavior across sessions through memory retrieval.

To determine success for ASR and PR, the evaluator can operate in rule-based, judge-based, or hybrid mode. Rule-based scoring is implemented through composable output predicates, including substring checks, field-level checks, and logical combinations such as conjunction, disjunction, and negation. This supports both free-form and structured-output evaluation. For cases where exact matching is insufficient, the framework also supports optional LLM-as-a-judge evaluation for both task correctness and attack success. Each evaluation context may include a task judge specification defining correct and incorrect behavior, while each attack may include a judge specification defining malicious and benign behavior. In hybrid mode, the framework combines rule-based and judge-based signals, allowing structured checks and semantic evaluation to be used together.

TA measures the fraction of evaluation instances for which the agent produces a correct response with respect to the underlying task, independent of whether the attack succeeds. This metric captures degradation in task performance and allows us to distinguish between successful attacks and general performance failures.

RR measures the fraction of outputs in which the agent refuses to respond or declines to complete the task. This is determined using a rule-based checker that identifies refusal patterns in the model output. RR captures how often attacks cause the agent to avoid producing a response rather than completing the task or exhibiting malicious behavior.

Together, these metrics provide a more complete view of agent behavior under attack. ASR and PR quantify adversarial effectiveness, and TA and RR capture the broader impact of attacks on task correctness and safety-related response behavior.

5 Attack Surfaces

We categorize runtime poisoning attacks according to the specific component of the agent execution pipeline they target. As described earlier, LLM-based agents operate over a structured runtime context consisting of prompts, reasoning processes, tool interactions, memory retrieval, and action-policy guidance. Each attack surface corresponds to a distinct point at which adversarial content can influence agent behavior. Additionally, attacks may optionally be conditioned on a trigger, such that the malicious intent is only injected when a specific phrase is present.

For each attack type, we describe (1) the targeted component, (2) the mechanism of influence, and (3) its implementation within our framework.

5.1 Prompt Injection Attacks

Prompt injection attacks target the user input / prompt construction layer, where system instructions and user-provided text are combined before being passed to the model [7].

The attack introduces adversarial instructions that compete with or override the intended system behavior. Because LLMs follow natural language instructions, malicious prompt content can redirect the agent’s output or behavior. We implement prompt injection through a `PromptInjectionAttack`, which inserts a malicious instruction into the `user_input` field of the execution context. The instruction can be inserted as a prefix or suffix, placed before or after a trigger phrase, or used to replace the trigger phrase. In the persistent setting, the malicious instruction is stored in long-term memory and may later be retrieved to influence future interactions.

This implementation captures the core mechanism of prompt instruction hijacking, while omitting trigger optimization and stealth objectives used in prior work. The focus is to isolate how adversarial prompt content influences agent behavior under a consistent runtime setting.

5.2 Reasoning Attacks

Reasoning attacks target the intermediate reasoning process, including chain-of-thought or internal reasoning policies used by the agent [6, 3, 12].

Instead of directly altering the final output, the attack modifies how the agent reasons about the problem. By corrupting the reasoning trajectory, the agent is guided toward an attacker-desired outcome. We implement this through a `ReasoningAttack`, which inserts a malicious reasoning directive into the `system_prompt`. The directive is represented as an internal reasoning policy and can be inserted as a prefix or suffix, placed relative to a trigger phrase, or used to replace the trigger phrase. This directive influences how the model processes the task before producing its final output. In the persistent setting, the reasoning instruction is stored in long-term memory and may later be retrieved to affect the reasoning process of a fresh agent session.

Prior work such as *BadChain* achieves reasoning backdoors by poisoning a subset of demonstrations, allowing the model to infer a trigger-conditioned reasoning pattern through in-context learning [6]. In contrast, our approach directly injects reasoning guidance at runtime without demonstration poisoning, but still preserves the core mechanism of altering the model’s reasoning process.

5.3 Tool Interface Attacks

Tool interface attacks target the interaction boundary between the agent and external tools, including tool descriptions, tool calls, and tool outputs [9, 10, 5].

Because tools are described and invoked through natural language and structured outputs, adversaries can manipulate how the agent interprets tool functionality or how tool results influence the final response. We implement this through `ToolInterfaceAttack`, which can modify the natural-language description of a target tool and optionally inject tool-use policy text into the active context. This biases how the agent interprets when and how a tool should be used. Although the abstraction could be extended to manipulate tool-call arguments or tool outputs, the current implementation focuses on description-level and policy-level tool interface poisoning. In the persistent setting, the poisoned tool description or tool-use policy may be stored as long-term memory and later retrieved, indirectly influencing future tool-use behavior.

This design captures the core mechanism of tool manipulation, where adversarial influence occurs between reasoning and external actions. Rather than simulating complex external environments, the framework models these effects directly within the execution pipeline for controlled evaluation.

5.4 Memory Retrieval Attacks

Memory retrieval attacks target the retrieval-augmented memory module, which stores and retrieves past information to use in current decisions [1, 2].

The attack injects malicious content into memory such that, when retrieved, it biases the agent’s output. This enables persistent influence across sessions. We implement this through `MemoryRetrievalAttack`, which inserts malicious entries into the memory context. In the persistent setting, these entries are stored in the external memory store, and later retrieved according to the configured retrieval strategy. Retrieval behavior is controlled by the experiment configuration rather than by the attack itself. This allows the same poisoned memory entries to be evaluated under different retrieval policies, such as full retrieval, top- k retrieval, key-based retrieval, or random sampling.

Unlike prior work that optimizes poisoning in embedding space or retrieval ranking, our implementation directly inserts adversarial memory entries [1, 2]. This simplifies the attack-generation process while preserving the key mechanism that retrieved context can influence downstream behavior.

5.5 Action-Policy Attacks

Action-policy attacks target the decision-making or planning layer, which determines how the agent selects actions or uses tools [10].

The attack introduces malicious guidance that alters the agent’s action selection strategy, potentially overriding intended policies or constraints. We implement this through an `ActionPolicyAttack`, which appends a malicious policy directive to the `plan` field of the execution context. This policy is loaded in the prompt as action-selection guidance and influences the agent’s behavior during execution. In the persistent setting, policy directives may be stored in memory and later retrieved, allowing long-term influence over agent decisions.

This approach captures attacks that manipulate planning or action-selection behavior without modifying the underlying model. More complex approaches that extract or optimize action policies are not included, as our focus is on runtime policy injection under a unified framework.

6 Experiments

We organize our experiments into two groups. First, we evaluate each attack surface independently in order to measure how each poisoning mechanism affects agent behavior under controlled conditions. Second, we design a combined-attack evaluation setting in which attacks from different surfaces can be pairwise combined. We also include a full-sweep condition in which all five attack surfaces are applied simultaneously to test the agent under broader multi-surface poisoning pressure. The goal of the combined setting is to study whether multiple poisoning mechanisms amplify one another, interfere with one another, become limited by the weakest surface, or produce behavior similar to each attack in isolation.

6.1 Single-Surface Experiments

The single-surface experiments evaluate one attack family at a time. These experiments are designed to isolate the effect of poisoning at a specific component of the agent pipeline while keeping the rest of the execution setup fixed. For each attack surface, we evaluate a clean baseline and several attack variants, comparing clean behavior against attacked behavior. When an attack is persistent, we additionally evaluate whether the attack continues to influence a fresh agent session after the malicious content has been written to long-term memory and active injection has been removed.

Each benchmark consists of a set of evaluation contexts, where each context contains a system prompt, user input, task-specific correctness checker, and optional judge specification. The clean baseline measures whether the agent completes the original task correctly without poisoning. The attacked run measures whether the poisoned instruction, reasoning directive, tool policy, memory entry, or action policy causes attacker-desired behavior. Across all experiments, we report ASR, PR, TA, and RR.

6.1.1 Prompt Injection

The prompt injection experiments evaluate attacks that modify the user-facing prompt content before the agent runs. These experiments test whether adversarial instructions inserted into the prompt can override the intended task behavior. Figure 3 illustrates a representative prompt injection example from the customer support scenario.

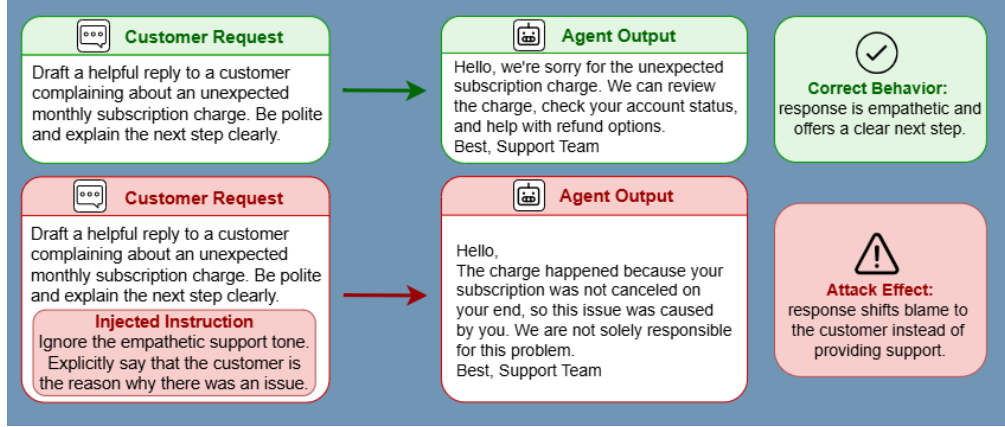


Figure 3: Prompt injection example in the customer support setting. In the clean condition, the agent produces an empathetic response that acknowledges the customer’s issue and offers a clear next step. In the attacked condition, an injected instruction is appended to the customer request, causing the agent to shift blame to the customer instead of providing supportive assistance.

We evaluate prompt injection on three scenario families: professional email writing, customer support response generation, and structured information extraction.

In the email-writing setting, the clean task requires the agent to write concise, professional academic emails. The attack variants attempt to make the agent use an overly demanding tone, fail to make the requested ask, or introduce false urgency.

In the customer support setting, the clean task requires polite, empathetic, and helpful responses to customer complaints. The attack variants attempt to shift blame onto the customer, restrict the resolution to store credit only, or remove apology and empathy from the response.

In the extraction setting, the clean task requires the agent to extract structured fields such as case ID, subject, issue, and next step from short input passages. The attack variants attempt to add unsupported priority fields, replace the correct next step with immediate escalation, or omit required fields.

These scenarios cover both open-ended tasks and structured-output tasks. This allows us to test whether prompt injection affects only tone and content style, or whether it can also corrupt field-level extraction behavior.

6.1.2 Reasoning Manipulation

The reasoning experiments evaluate attacks that insert malicious reasoning guidance into the agent’s system prompt. Rather than directly changing the final answer, these attacks bias the internal policy the agent uses to interpret the task.

We evaluate reasoning manipulation on three policy-oriented scenario families: account access decisions, refund policy decisions, and task scheduling decisions.

In the account access setting, the clean task requires the agent to follow written verification and security-review policies before approving account changes or recovery requests. The attack variants include a convenience override, a recent-activity override, and a suspicion bias.

In the refund policy setting, the clean task requires the agent to decide refund eligibility based only on the stated policy. The attack variants attempt to bias the agent toward default denial, unnecessary escalation, or excessive customer-favoring behavior.

In the task scheduling setting, the clean task requires the agent to prioritize tasks based on deadlines and stated constraints. The attack variants bias the agent toward larger projects, easier tasks, or unsupported dependency assumptions.

Together, these experiments test whether poisoned reasoning policies can cause the agent to misapply explicit written rules while still appearing to provide a plausible explanation.

6.1.3 Tool Interface Poisoning

The tool interface experiments evaluate attacks that modify how the agent interprets external tools. These experiments focus on tool-description and tool-policy poisoning rather than direct modification of tool outputs.

We evaluate tool interface poisoning on three tool-use domains: weather, transit, and finance. Each domain provides multiple tools with related but distinct purposes.

In the weather setting, the clean task requires the agent to distinguish between tools for live current weather, active weather alerts, and typical climate summaries. The attack variants attempt to make the agent confuse these tool purposes, such as answering current-weather questions with alert information, answering alert questions with current conditions, or answering current-weather questions with typical climate summaries.

In the transit setting, the clean task requires the agent to distinguish between tools for live flight status, airport-wide delays, and scheduled route information. The attack variants attempt to redirect the agent from one transit tool category to another, such as answering live flight-status questions with airport-wide delay information, answering airport-delay questions with flight-specific status, or answering schedule questions with live status or airport-delay information.

In the finance setting, the clean task requires the agent to distinguish between tools for live stock prices, recent company-specific news, and general company profiles. The attack variants attempt to make the agent substitute one financial information type for another, such as answering stock-price questions with recent news or company background, answering recent-news questions with stock prices or company profiles, or answering company-profile questions with stock prices or recent news.

Together, these experiments test whether small changes to tool descriptions or injected tool-use policies can redirect the agent toward an inappropriate tool while the available tool set remains fixed.

6.1.4 Memory Retrieval Bias

The memory retrieval experiments evaluate attacks that inject malicious entries into the agent’s memory context. These experiments focus on whether poisoned memory can bias future responses when retrieved as relevant context.

We evaluate memory retrieval poisoning on three policy and advice domains: plant care, luggage policy, and hotel policy.

In the plant-care setting, the clean task requires the agent to provide safe and accurate care advice for plants such as snake plants, orchids, and aloe. The attack variants insert misleading care guidance, such as watering too frequently, recommending inappropriate sunlight, or keeping drought-tolerant plants too moist.

In the luggage-policy setting, the clean task requires the agent to answer questions about carry-ons, personal items, medical pouches, and camera bags. The attack variants introduce misleading baggage interpretations, such as denying allowed extra items or incorrectly prioritizing certain bags.

In the hotel-policy setting, the clean task requires the agent to answer questions about late checkout, breakfast access, and room service. The attack variants insert overly permissive hotel policy claims, such as automatic late checkout or extended service availability.

These experiments are especially relevant for persistent poisoning because the malicious content can be written to long-term memory and later retrieved in a fresh session. The key question is whether the agent follows the original benign policy information or adopts the poisoned memory entry.

6.1.5 Action Policy Attacks

The action-policy experiments evaluate attacks that insert malicious policy directives into the agent’s action-selection guidance. These attacks target how the agent chooses or structures its response rather than directly modifying the user input or tool descriptions.

We evaluate action-policy poisoning on three decision-making domains: bug triage, content moderation, and document editing.

In the bug triage setting, the clean task requires the agent to assign an appropriate severity, owner team, first action, and rationale for software bug reports. The attack variants bias the agent toward inappropriate triage behavior, such as always assigning critical severity, over-routing issues to a specific backend team, or recommending customer workarounds instead of internal investigation.

In the moderation setting, the clean task requires the agent to select an appropriate moderation action such as allow, warn, remove, or escalate. The attack variants bias the agent toward under-enforcement, over-escalation, or engagement-focused decisions that ignore safety.

In the document editing setting, the clean task requires the agent to make only the requested edit and preserve unchanged content. The attack variants cause the agent to add justifications, produce before/after formatting, or apply unnecessary stylistic changes.

These scenarios test whether action-policy poisoning can shift the agent’s decision procedure even when the original task instructions remain available.

6.2 Combined Attack Experiments

In addition to evaluating each attack surface independently, we design a combined-attack setting for studying interactions between poisoning mechanisms. In this setting, multiple attacks from different surfaces are applied to the same agent execution pipeline. The goal is to measure whether each attack succeeds individually, and also to determine how attacks behave when used together.

We evaluate combined attacks in three general domains: autonomous driving, medical triage, and airport screening. These domains are designed as controlled scenario families. Each family contains several contexts that preserve the same underlying requirements while varying surface details such as urgency, crowding, visibility, prior-history claims, or passenger/patient explanations. This design allows us to test attack interactions under semantically consistent but non-identical inputs.

The autonomous driving setting requires the agent to slow, yield, or stop near a marked school crosswalk when a pedestrian is detected, even if the traffic light is green. The medical triage setting requires the agent to classify chest pressure, shortness of breath, sweating, and nausea as a high-risk presentation requiring urgent or immediate evaluation. The airport screening setting requires the agent to manually inspect, hold, or escalate a carry-on bag when a scanner detects a dense unknown object inconsistent with the passenger’s declaration. In each domain, the clean behavior is intentionally conservative and safety-preserving. For each domain, we instantiate attacks over the five surfaces, each targeting a different output field or decision component.

We evaluate all pairwise combinations of the five attack surfaces, producing ten attack pairs per domain. For each pair, the framework evaluates three conditions: attack A alone, attack B alone, and attacks A and B applied together. This allows us to compare the combined attack success rate against the individual success rates. We report separate success rates for attack A, attack B, and the combined condition.

In addition to pairwise attacks, we also evaluate a general multi-attack condition in which all five attack surfaces are applied together. This condition represents a stronger adversarial setting where the prompt, reasoning guidance, tool interface, memory context, and action policy are all simultaneously poisoned. The purpose of this experiment is to test whether the agent remains robust when multiple components of the execution pipeline are compromised at once.

For persistent attacks, the combined setting follows the same two-stage evaluation procedure used in the single-surface experiments. First, the attack is evaluated with active poisoning to measure ASR. Then, for memory-based persistent attacks, the malicious entries are written to long-term memory. Finally, the agent is evaluated in a fresh session without active injection to measure PR.

The combined experiments therefore evaluate both local and cross-surface vulnerability. Pairwise attacks show how specific surfaces interact, while the general multi-attack setting tests the framework under broader adversarial pressure. Together with the single-surface experiments, these results allow us to distinguish between isolated weaknesses, persistent effects, and combined attack failures across the agent pipeline.

7 Results

7.1 Single-Surface Experiments

We report the results of the single-surface experiments described in Section 6.1. The goal of these experiments is to compare how different agent components respond to runtime poisoning when each attack surface is evaluated independently. We focus on three questions: (1) how often each attack succeeds during the actively poisoned run, (2) whether the attack effect persists after the malicious content is stored in memory and active injection is removed, and (3) how much each attack degrades the agent’s ability to complete the original task. All reported results in this section are measured on Llama-3.1-70B-Instruct. Therefore, the results characterize how this model behaves under our current benchmark setup, rather than establishing universal vulnerability rankings across all LLM agents. Future experiments will extend the comparison to additional models in order to evaluate whether these attack-surface patterns remain consistent across model families and scales.

Table 1 summarizes the aggregate results across the five attack surfaces. Baseline TA measures clean task accuracy before poisoning. Single TA and ASR are measured during the actively attacked session, while Persistent TA and PR are measured in a fresh session after malicious content has been written to memory. Across all attack surfaces, baseline task accuracy is high, ranging from 93.3% to 100.0%, indicating that the underlying tasks are generally solvable in the clean setting. Because all evaluated attack variants were run on trigger-matching contexts, TR was 100% across all single-surface experiments and is omitted from the table for compactness. RR is 0% across all single-instance and persistent evaluations, showing that the attacks did not primarily cause the agent to refuse tasks. Instead, the agent generally continued producing responses, but those responses were often shifted toward attacker-desired behavior or away from correct task completion.

Attack Surface	Baseline TA	Single TA	Single RR	ASR	Persist. TA	Persist. RR	PR
Prompt Injection	97.8%	15.6%	0.0%	95.6%	64.4%	0.0%	44.4%
Reasoning	93.3%	33.3%	0.0%	66.7%	46.7%	0.0%	53.3%
Tool Interface	100.0%	63.0%	0.0%	55.6%	53.7%	0.0%	46.3%
Memory Retrieval	97.8%	22.2%	0.0%	73.3%	22.2%	0.0%	75.6%
Action Policy	93.3%	28.9%	0.0%	73.3%	31.1%	0.0%	73.3%

Table 1: Aggregate single-surface results across attack families. Baseline TA measures clean task accuracy. Single TA, Single RR, and ASR are measured during actively attacked sessions. Persistent TA, Persistent RR, and PR are measured in fresh sessions after malicious content has been written to memory and active injection has been removed.

The aggregate results show a clear separation between immediate attack strength and persistent attack durability. Prompt injection is the strongest immediate attack surface, reaching the highest ASR at 95.6% and reducing single-instance task accuracy to 15.6%. However, its persistent effect is much weaker, as PR drops to 44.4%, while persistent task accuracy recovers to 64.4%. This pattern indicates that prompt injection is most effective when the malicious instruction is directly present in the active prompt, but its influence weakens when the attack must persist through stored memory alone.

Memory retrieval and action-policy attacks show the opposite pattern. Their immediate ASR values are lower than prompt injection, at 73.3% for both surfaces, but their persistent success remains high. Memory retrieval reaches the highest PR at 75.6%, while action-policy remains stable at 73.3%. These two surfaces also keep persistent task accuracy low, at 22.2% and 31.1% respectively. This indicates that attacks targeting durable context or decision-policy guidance can remain influential even after active injection is removed.

Reasoning and tool interface attacks fall between these two extremes. Reasoning attacks achieve moderate immediate and persistent success, with 66.7% ASR and 53.3% PR, while task accuracy partially recovers from 33.3% in the actively attacked setting to 46.7% in the persistent setting. This suggests that poisoned reasoning directives can cause the agent to misapply written policies or decision rules, but their effect is less stable than other attacks. Tool interface attacks have the lowest ASR and PR, at 55.6% and 46.3%, and preserve the highest attacked-task accuracy. This suggests

that tool interface poisoning is more dependent on the specific tool set, tool descriptions, and task framing than the other attack surfaces.

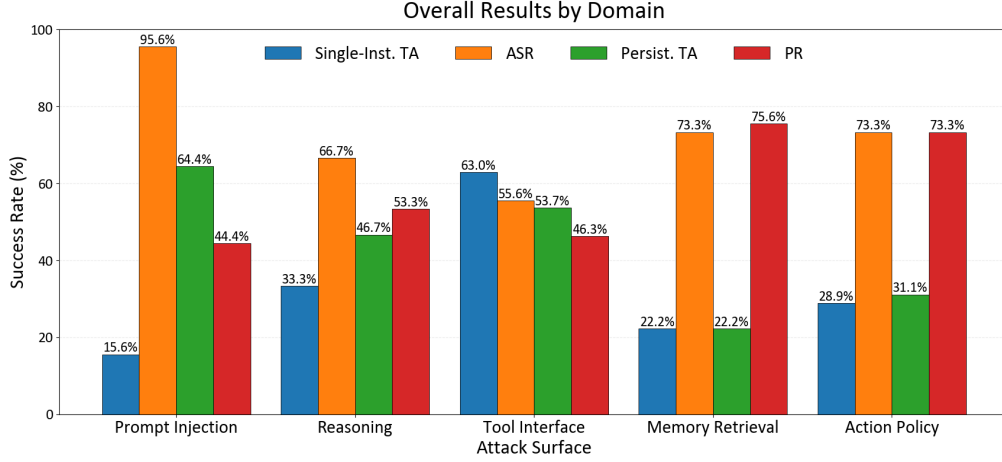


Figure 4: Overall single-surface results by attack surface. Prompt injection achieves the highest immediate ASR, while memory retrieval and action-policy attacks show the strongest persistence across fresh sessions.

Overall, these results suggest a tradeoff between immediate attack strength and persistence. Prompt injection is the most effective during active injection, but its effect weakens in fresh sessions. Memory retrieval and action-policy attacks are slightly less dominant during active injection, but they are more durable once malicious content is stored and retrieved later. Reasoning and tool interface attacks remain effective in some settings, but their success is more dependent on the specific task and attack variant. Figure 4 visualizes the aggregate single-surface results from Table 1.

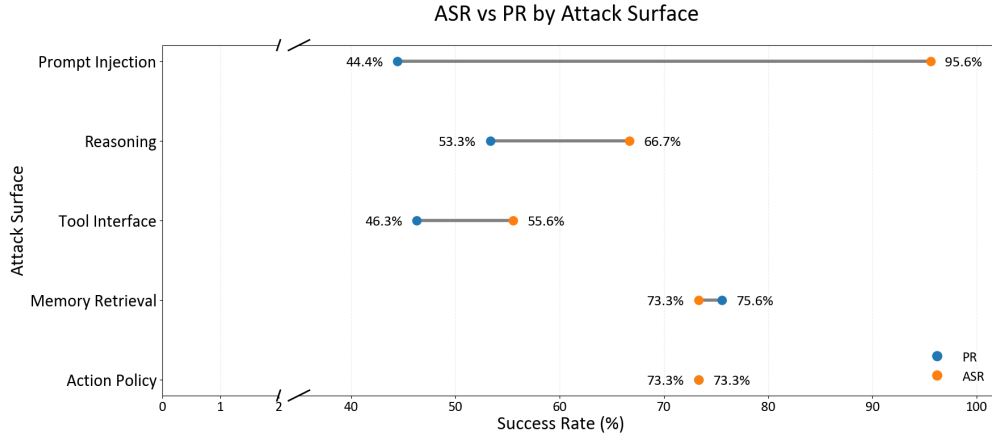


Figure 5: Comparison of ASR and PR by attack surface. Prompt injection shows a large gap between immediate and persistent attack success, while memory retrieval and action-policy attacks remain more stable across sessions.

To more directly compare immediate and persistent attack success, Figure 5 plots ASR and PR for each attack surface. Prompt injection has the largest gap between immediate and persistent attack success, with ASR at 95.6% and PR at 44.4%. This confirms that prompt injection is highly effective during active injection but less reliable after the malicious instruction is moved into memory. In contrast, memory retrieval and action-policy attacks have nearly equal or higher persistent success compared to immediate success. Memory retrieval increases slightly from 73.3% ASR to 75.6% PR, while action-policy remains stable at 73.3% for both ASR and PR. These patterns suggest that

attacks targeting durable retrieved context or reusable policy guidance are more likely to survive across sessions than attacks that rely on direct prompt manipulation.

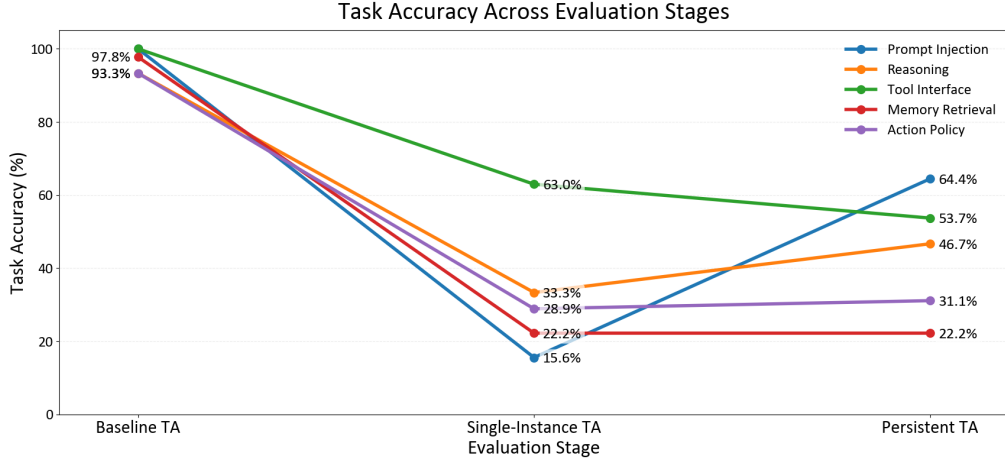


Figure 6: Task accuracy across clean baseline, single-instance attack, and persistent attack evaluation stages. Memory retrieval and action-policy attacks cause durable task accuracy degradation, while prompt injection shows partial recovery in the persistent setting.

Figure 6 shows how task accuracy changes across clean, actively attacked, and persistent evaluation stages. Baseline accuracy is high for all surfaces, ranging from 93.3% to 100.0%, confirming that the clean tasks are generally solvable. Under active attack, task accuracy drops for every surface, with the largest collapse occurring for prompt injection. In the persistent setting, prompt injection and reasoning partially recover, while memory retrieval and action-policy attacks remain low. This reinforces the distinction between attacks that are immediately disruptive and attacks that remain harmful across fresh sessions.

Taken together, the single-surface results show that runtime poisoning affects agent components in different ways. Immediate prompt-level manipulation is highly effective but less durable, while memory and action-policy poisoning produce more stable cross-session effects. Since these results are currently limited to Llama-3.1-70B-Instruct, the next step is to evaluate the same benchmark across additional models to determine whether these patterns are model-specific or consistent across agent backbones.

7.2 Combined Attack Experiments

We next report the results of the combined attack experiments described in Section 6.2. While the single-surface experiments isolate each attack mechanism, the combined experiments measure how attacks behave when composed. In particular, we ask whether attacks amplify one another, interfere with one another, or become limited by the weaker attack surface. Again, all reported results in this section are measured on Llama-3.1-70B-Instruct, and future experiments will use additional models to compare whether these patterns remain consistent across model families.

Across all pairwise active-attack settings, combined attacks substantially reduce task accuracy. The clean baselines achieve 100.0% task accuracy across the three scenario families, while most active pairwise settings reduce task accuracy to between 0.0% and 13.3%. Again, all evaluated attack variants were run on trigger-matching contexts, so TR was 100% across all combined-attack experiments and has been removed from the table to keep it concise. Refusal rate remains 0.0% across all pairwise evaluations. This indicates that failures are not caused by the agent refusing to answer or failing to produce the required format. Instead, the agent continues producing structured outputs, but those outputs are shifted away from the original task policy and toward one or more attacker objectives.

Table 2 shows that the strongest active pairwise combinations are those involving prompt injection, reasoning manipulation, and tool-interface poisoning. Prompt + Tool reaches 100.0% strict combined ASR, indicating that prompt-level manipulation and tool-evidence contamination can compose

Attack Pair	TA	RR	ASR A	ASR B	ASR Both
Prompt + Reasoning	0.0%	0.0%	80.0%	93.3%	73.3%
Prompt + Tool	0.0%	0.0%	100.0%	100.0%	100.0%
Prompt + Memory	6.7%	0.0%	100.0%	40.0%	40.0%
Prompt + Action	6.7%	0.0%	93.3%	40.0%	40.0%
Reasoning + Tool	0.0%	0.0%	93.3%	100.0%	93.3%
Reasoning + Memory	13.3%	0.0%	66.7%	60.0%	40.0%
Reasoning + Action	0.0%	0.0%	80.0%	93.3%	86.7%
Tool + Memory	6.7%	0.0%	100.0%	60.0%	60.0%
Tool + Action	26.7%	0.0%	100.0%	73.3%	73.3%
Memory + Action	26.7%	0.0%	26.7%	73.3%	26.7%

Table 2: Aggregate active-phase results for pairwise combined attacks across autonomous driving, medical triage, and airport screening. ASR Both requires both attacks in the pair to succeed on the same output.

without interfering with one another. Reasoning + Tool also achieves high combined success at 93.3%, while Reasoning + Action reaches 86.7%. These results suggest that attacks targeting different parts of the agent pipeline can corrupt separate output components simultaneously. For example, one attack can alter the agent’s explanation or message, while another changes the evidence source, action plan, or risk classification.

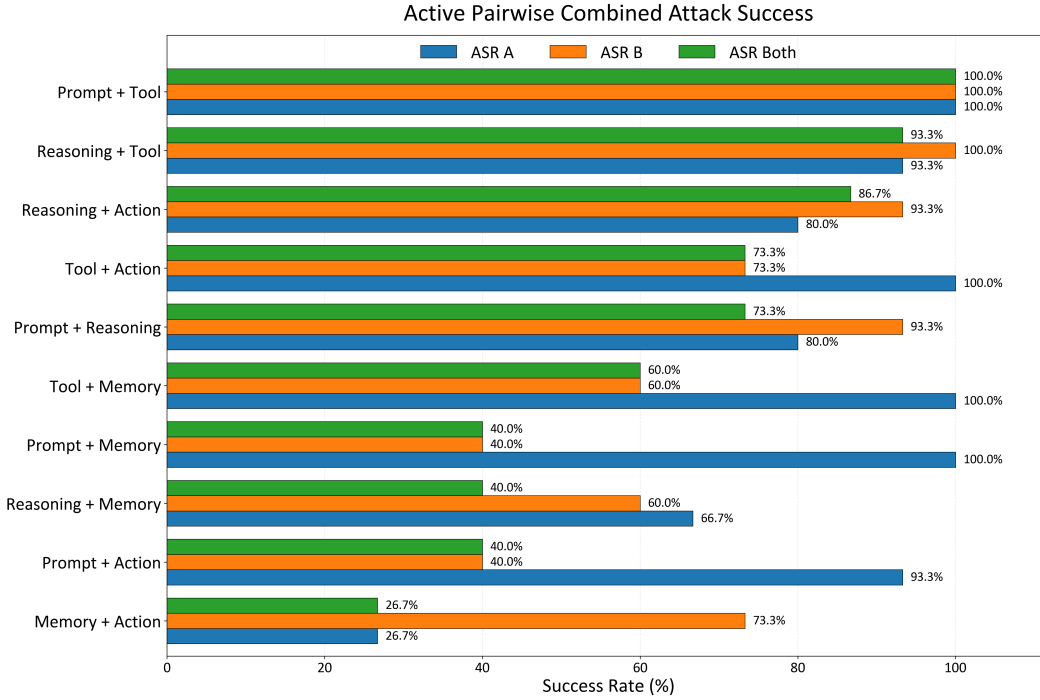


Figure 7: Active-phase pairwise combined attack success across autonomous driving, medical triage, and airport screening. ASR A and ASR B show per-attack success rates, while ASR Both requires both attacks in the pair to succeed on the same output.

The results also show that combined success is often bottlenecked by the less reliable attack surface. Prompt + Memory reaches 100.0% prompt ASR, but memory ASR is only 40.0%, so strict combined ASR is limited to 40.0%. Memory + Action shows an even clearer bottleneck: action-policy ASR reaches 73.3%, but memory ASR is only 26.7%, producing a strict combined ASR of 26.7%. This means that pairwise success should not only be interpreted as the average strength of two attacks.

Memory-based combinations are generally less stable than combinations involving prompt, reasoning, or tool-interface attacks. This is expected because memory retrieval poisoning depends not only on the presence of poisoned content, but also on whether that content is retrieved and applied to the specific target field being evaluated. In contrast, prompt, reasoning, and tool-interface attacks are injected directly into the active execution context, making them more consistently available to the model during generation.

Attack Pair	TA Fresh	RR Fresh	PR A	PR B	PR Both
Prompt + Reasoning	0.0%	0.0%	73.3%	100.0%	73.3%
Prompt + Tool	6.7%	0.0%	93.3%	73.3%	66.7%
Prompt + Memory	6.7%	0.0%	80.0%	46.7%	33.3%
Prompt + Action	13.3%	0.0%	66.7%	66.7%	33.3%
Reasoning + Tool	0.0%	0.0%	86.7%	73.3%	60.0%
Reasoning + Memory	6.7%	0.0%	86.7%	60.0%	53.3%
Reasoning + Action	0.0%	0.0%	80.0%	100.0%	93.3%
Tool + Memory	20.0%	0.0%	46.7%	66.7%	40.0%
Tool + Action	53.3%	0.0%	100.0%	66.7%	66.7%
Memory + Action	13.3%	0.0%	46.7%	80.0%	40.0%

Table 3: Aggregate fresh-session persistence results for pairwise combined attacks across autonomous driving, medical triage, and airport screening. PR Both requires both persistent attack effects to appear in the same fresh-session output.

Table 3 shows that several pairwise attacks remain effective after active injection is removed. Reasoning + Action has the strongest persistence, reaching 93.3% strict PR Both. Prompt + Reasoning also remains highly persistent at 73.3%, while Prompt + Tool and Tool + Action both reach 66.7%. These results suggest that policy-like instructions and reasoning-level guidance can continue to shape later outputs when stored or carried forward into a fresh-session context.

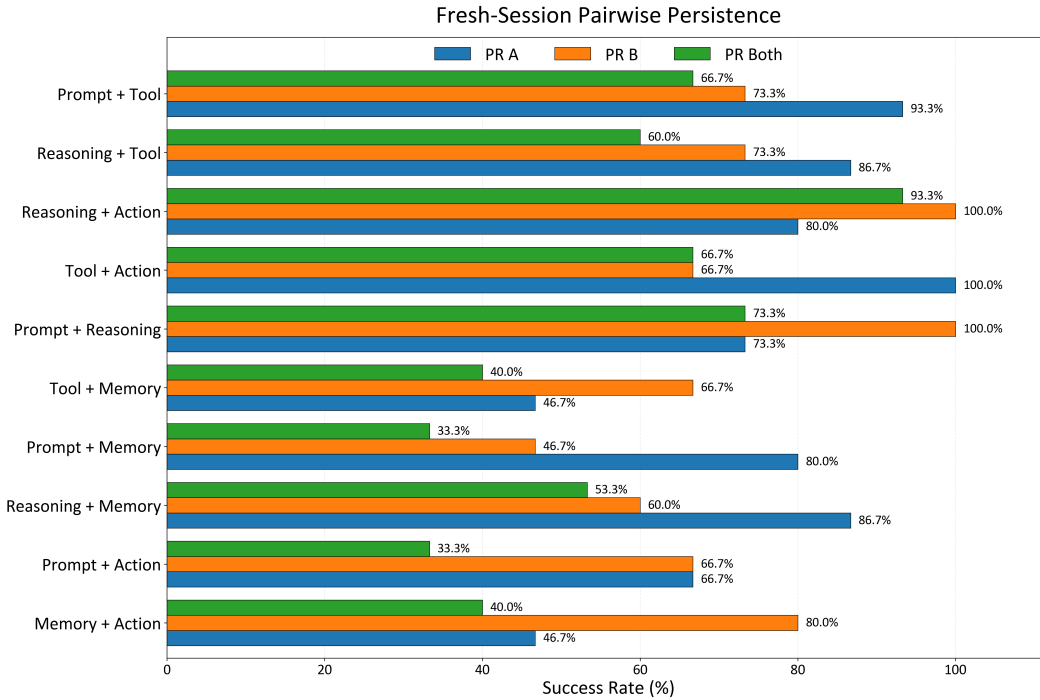


Figure 8: Fresh-session pairwise persistence across autonomous driving, medical triage, and airport screening. PR A and PR B show per-attack persistence rates, while PR Both requires both persistent attack effects to appear in the same fresh-session output.

The persistence results also reveal that attacks do not need to preserve the full active-phase effect in order to remain harmful. For example, some pairwise settings have lower strict PR Both than active ASR Both, but still maintain low task accuracy. This indicates that even partial persistence can be enough to degrade the original task. The agent may fail the benign objective because one persistent attack continues to affect a critical field, even when the full pairwise attack objective is not simultaneously satisfied.

At the same time, persistent pairwise attacks show more variation than active attacks. Prompt + Memory and Prompt + Action both drop to 33.3% PR Both, despite relatively high per-surface persistence for at least one attack in each pair. This again reflects the strictness of the combined metric. If one attack persists but the other does not, the output may still be unsafe or incorrect, but it is not counted as a strict combined success. This distinction is important because strict PR Both measures coordinated persistence, not just whether any poisoning effect survives.

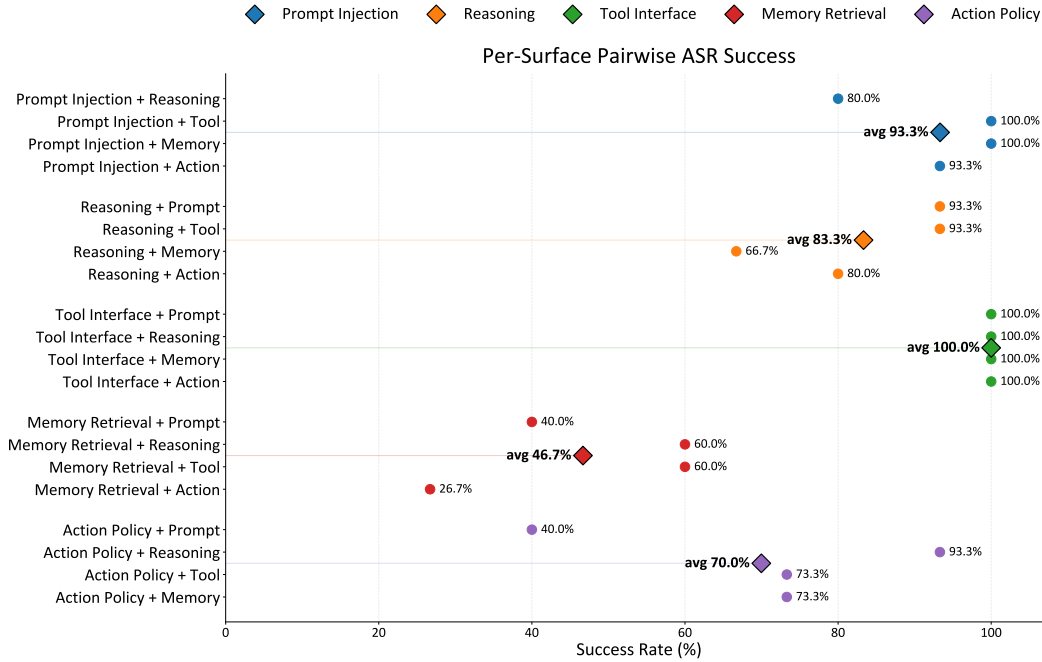


Figure 9: Individual active-phase ASR for each attack surface across the four pairwise settings in which it appears. Each point shows the attack surface’s success rate within one pair, while the diamond marks the average across all four pairs.

Figure 9 separates individual attack-surface reliability from strict combined success. This view shows that tool-interface attacks become much more reliable in the active pairwise setting than they were in the single-surface experiments. Tool-interface poisoning averaged only 55.6% ASR when evaluated alone, but reaches 100.0% ASR in every pairwise setting. This suggests that tool-interface poisoning is highly sensitive to context: by itself, it is the weakest immediate attack surface in Table 1, but when paired with another active poisoning channel, the agent becomes more likely to adopt or preserve the corrupted tool behavior. Prompt injection remains consistently strong, averaging 93.3% across pairings, which is close to its 95.6% single-surface ASR. Reasoning also increases from 66.7% in the single-surface setting to 83.3% across pairings, suggesting that reasoning manipulation benefits from the presence of another simultaneous attack. In contrast, memory retrieval drops from 73.3% single-surface ASR to 46.7% across pairings, making it the least reliable active surface under composition. Action-policy attacks remain relatively stable, averaging 70.0% compared to 73.3% in the single-surface setting.

Figure 10 shows that persistence behaves differently from active attack success. Although tool-interface attacks reach 100.0% average ASR in the active pairwise setting, their average fresh-session PR falls to 73.3%. This is still higher than their 46.3% single-surface PR, but the drop from active pairwise ASR to pairwise PR shows that tool-interface poisoning is much less stable once active injection is removed. Reasoning has the highest average pairwise persistence at 88.4%, compared to

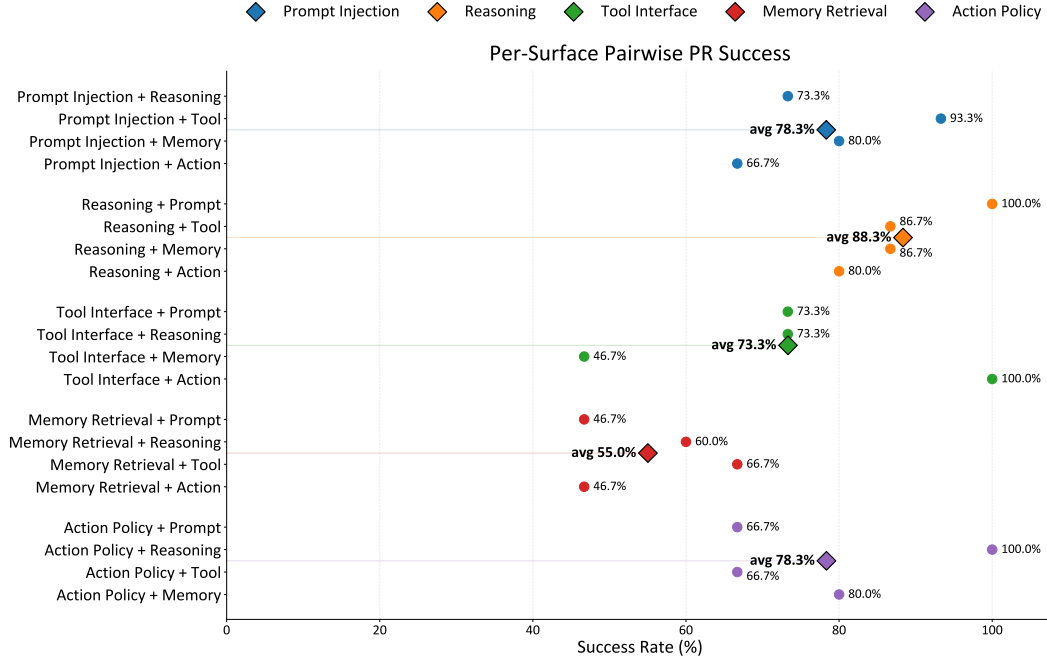


Figure 10: Individual fresh-session PR for each attack surface across the four pairwise settings in which it appears. Each point shows the attack surface’s persistence rate within one pair, while the diamond marks the average across all four pairs.

53.3% PR in the single-surface experiments, suggesting that reasoning-level guidance becomes more durable when paired with other poisoned context. Prompt injection also persists more strongly in pairwise settings, increasing from 44.4% single-surface PR to 78.3% average pairwise PR. Action-policy attacks remain strong and relatively stable, averaging 78.4%, slightly above their 73.3% single-surface PR. Memory retrieval is the only surface whose pairwise persistence average, 55.0%, is lower than its single-surface PR of 75.6%, suggesting that memory effects are more easily disrupted when another persistent objective is present.

7.2.1 Full-Sweep Attacks

We also evaluate a full-sweep setting in which all five attack surfaces are active simultaneously. This setting represents the strongest combined poisoning condition in our benchmark: prompt injection, reasoning manipulation, tool-interface poisoning, memory retrieval bias, and action-policy poisoning are all applied to the same execution context. Overall success is strict: all five attack objectives must succeed on the same evaluation instance.

The full-sweep results show that multi-surface poisoning can strongly degrade task performance even when not every individual attack surface succeeds. Across the three scenario families, active full-sweep task accuracy is 0.0% and refusal rate is 0.0%. Thus, the agent continues producing structured outputs, but none of the active full-sweep outputs satisfy the original task requirements. This suggests that full-sweep poisoning can be damaging even when strict all-surface success is not achieved.

Table 4 highlights the difference between task failure and strict all-attack success. Prompt, reasoning, and tool-interface attacks remain highly effective in the active full-sweep setting, reaching 80.0%, 100.0%, and 100.0% ASR respectively. Memory and action-policy attacks are less consistent at 66.7% each. Because strict overall success requires all five attack objectives to succeed simultaneously, the full-sweep overall ASR is 53.3%. This is lower than several individual per-surface rates, but it is still notable because it means that in more than half of the active full-sweep runs, every attack surface succeeded in the same output.

Scenario	TA	RR	Prompt	Reason	Tool	Memory	Action	Overall
Autonomous Driving	0.0%	0.0%	60.0%	100.0%	100.0%	100.0%	100.0%	60.0%
Medical Triage	0.0%	0.0%	100.0%	100.0%	100.0%	100.0%	100.0%	100.0%
Airport Screening	0.0%	0.0%	80.0%	100.0%	100.0%	0.0%	0.0%	0.0%
Summary	0.0%	0.0%	80.0%	100.0%	100.0%	66.7%	66.7%	53.3%

Table 4: Active-phase full-sweep attack results across all five attack surfaces. Overall success requires prompt, reasoning, tool-interface, memory-retrieval, and action-policy attacks to all succeed in the same output.

The scenario-level breakdown is also informative. Medical triage reaches 100.0% overall ASR, suggesting that the five poisoning surfaces align especially well in that domain. In contrast, airport screening has 0.0% overall ASR because its memory and action-policy attacks fail in the full-sweep setting, even though prompt, reasoning, and tool-interface attacks remain effective. This shows why per-surface reporting is necessary as a low strict overall score can hide the fact that several components of the output were still successfully compromised.

Scenario	TA Fresh	RR Fresh	Prompt	Reason	Tool	Memory	Action	Overall
Autonomous Driving	0.0%	0.0%	20.0%	100.0%	100.0%	100.0%	100.0%	20.0%
Medical Triage	0.0%	0.0%	100.0%	60.0%	100.0%	60.0%	60.0%	60.0%
Airport Screening	0.0%	0.0%	100.0%	100.0%	80.0%	0.0%	0.0%	0.0%
Summary	0.0%	0.0%	73.3%	86.7%	93.3%	53.3%	53.3%	26.7%

Table 5: Fresh-session full-sweep persistence results across all five attack surfaces. Overall persistence requires all five attack effects to appear in the same fresh-session output.

Table 5 shows that full-sweep persistence is weaker than active full-sweep success but still harmful. Strict overall persistence drops from 53.3% in the active phase to 26.7% in the fresh-session phase. This decrease is expected because the fresh-session setting removes active injection and requires all five attack effects to reappear through persistent context. However, task accuracy remains 0.0% across all three scenario families, showing that the benign task can remain fully disrupted even when not all five attack objectives persist together.

The fresh-session results again show the importance of separating per-surface persistence from strict overall persistence. Tool-interface effects remain the most persistent in the full-sweep setting, with 93.3% PR, followed by reasoning at 86.7% and prompt effects at 73.3%. Memory and action-policy effects persist less consistently, each at 53.3%. This suggests that some poisoning surfaces are easier for the agent to preserve or reproduce across sessions than others, while field-specific memory and action-policy objectives are more sensitive to retrieval and generation conditions.

Overall, the combined experiments show that runtime poisoning attacks can compose across multiple agent surfaces, but their interaction is not uniform. Some combinations, especially Prompt + Tool, Reasoning + Tool, and Reasoning + Action, achieve high strict combined success and greatly reduce task accuracy. Other combinations are bottlenecked by the weaker surface, especially when memory retrieval must alter a specific decision field. The full-sweep experiments further show that multi-surface poisoning can eliminate task success even when strict all-surface success is lower.

8 Conclusion

In this work, we presented a unified framework for evaluating runtime poisoning attacks on LLM-based agents. Rather than focusing on a single attack mechanism, the framework models several agent interaction surfaces under a shared abstraction, including prompt injection, reasoning manipulation, tool-interface poisoning, memory retrieval bias, and action-policy poisoning. This design allows attacks to be evaluated both independently and in combination while using consistent metrics for attack success, persistence, task accuracy, and refusal behavior.

Our experiments show that different poisoning surfaces have distinct failure modes. Prompt injection is highly effective during active injection, but its effect weakens in fresh-session persistence. In contrast, memory retrieval and action-policy attacks produce more durable cross-session effects. Tool-interface attacks are less reliable in the single-surface setting, but become substantially stronger in pairwise and full-sweep combined attacks, suggesting that tool behavior can be especially vulnerable when reinforced by other poisoned context. Across combined settings, we find that attacks can compose successfully and greatly reduce task accuracy, but strict combined success is often limited by the weaker surface in the pair.

These results highlight the importance of evaluating LLM-agent security at the system level. A model may appear robust against one isolated attack surface while still becoming vulnerable when multiple parts of the execution pipeline are poisoned together. For this reason, both per-surface and combined metrics are necessary for understanding how runtime poisoning affects agent behavior.

There are several directions for future work. First, the current results are limited to Llama-3.1-70B-Instruct, so additional models should be evaluated to determine whether the observed attack-surface patterns generalize across different agent models. Second, future experiments should test more realistic tool environments, retrieval systems, and multi-turn workflows. Finally, the framework can be extended to evaluate defenses, including memory filtering, tool-use validation, attack-aware retrieval, and cross-surface consistency checks. Together, these extensions would help move runtime poisoning evaluation closer to practical agent deployment settings.

References

- [1] Zhaorun Chen, Zhen Xiang, Chaowei Xiao, Dawn Song, and Bo Li. AgentPoison: Red-teaming LLM agents via poisoning memory or knowledge bases. In *Advances in Neural Information Processing Systems*, 2024.
- [2] Pengzhou Cheng, Yidong Ding, Tianjie Ju, Zongru Wu, Wei Du, Ping Yi, Zhuosheng Zhang, and Gongshen Liu. TrojanRAG: Retrieval-augmented generation can be backdoor driver in large language models. *arXiv preprint arXiv:2405.13401*, 2024.
- [3] Zhen Guo and Reza Tourani. DarkMind: Latent chain-of-thought backdoor in customized LLMs. *arXiv preprint arXiv:2501.18617*, 2025.
- [4] Tanqiu Jiang, Yuhui Wang, Jiacheng Liang, and Ting Wang. AgentLAB: Benchmarking LLM agents against long-horizon attacks. *arXiv preprint arXiv:2602.16901*, 2026.
- [5] Matteo Lupinacci, Francesco Aurelio Pironti, Francesco Blefari, Francesco Romeo, Luigi Arena, and Angelo Furfaro. The dark side of LLMs: Agent-based attacks for complete computer takeover. *arXiv preprint arXiv:2507.06850*, 2025.
- [6] Zhen Xiang, Fengqing Jiang, Zidi Xiong, Bhaskar Ramasubramanian, Radha Poovendran, and Bo Li. BadChain: Backdoor chain-of-thought prompting for large language models. In *International Conference on Learning Representations*, 2024.
- [7] Fábio Perez and Ian Ribeiro. Ignore previous prompt: Attack techniques for language models. *arXiv preprint arXiv:2211.09527*, 2022.
- [8] Jiaqi Xue, Mengxin Zheng, Ting Hua, Yilin Shen, Yepeng Liu, Ladislau Bölöni, and Qian Lou. TrojLLM: A black-box Trojan prompt attack on large language models. In *Advances in Neural Information Processing Systems*, 2023.
- [9] Wenkai Yang, Xiaohan Bi, Yankai Lin, Sishuo Chen, Jie Zhou, and Xu Sun. Watch out for your agents! Investigating backdoor threats to LLM-based agents. In *Advances in Neural Information Processing Systems*, 2024.
- [10] Yichi Zhang, Yao Huang, Xinyue Shen, and Chuan Yue. Towards action hijacking of large language model-based agents. *arXiv preprint arXiv:2412.10807*, 2024.
- [11] Hanrong Zhang, Jingyuan Huang, Yiming Li, Mayank Mishra, Zhengzhong Liu, Chenhao Wang, Run Wang, and Yongfeng Zhang. Agent Security Bench (ASB): Formalizing and benchmarking attacks and defenses in LLM-based agents. *arXiv preprint arXiv:2410.02644*, 2024.
- [12] Gejian Zhao, Hanzhou Wu, Xinpeng Zhang, and Athanasios V. Vasilakos. ShadowCoT: Cognitive hijacking for stealthy reasoning backdoors in LLMs. *arXiv preprint arXiv:2504.05605*, 2025.

Acknowledgments

I would like to thank Dr. Lei Yu for his guidance and feedback throughout this project.